

CCP230
Desenvolvimento de Algoritmos

Profa. Dra. Gabriela Oliveira Biondi

Departamento de Ciência da Computação FEI

FEI JESUÍTAS BRASIL



Strings

Aula 07

O tipo *char*

- O que é o tipo ***char***?
  - Representa 1 byte(8 bits)
  - Ou seja, um char representa um valor entre 0 e 255 (2^8)
  - Normalmente utilizado para representar caracteres

O tipo *char*

- O que é o tipo ***char***?
  - A representação de um caractere é dada através de um número inteiro
  - Esse número segue um padrão conhecido entre diversos sistemas computacionais:
    - ASCII - American Standard Codefor Information Interchange
    - UTF - Unicode Transformation Format

Tabela ASCII

- Tabela **ASCII**:
  - 7 bits (números de 0 a 127)
- Tabela **ASCII** Estendida
  - 8 bits
  - Igual à tabela ASCII, porém contém mais caracteres
  - Além do 0 ao 127, contém do 128 até o 255 (inclui alguns caracteres com acentos)

Tabela ASCII

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Tabela ASCII

- Como consultar a tabela em código (Linguagem C)?

```c
int caractere = 97;
printf("%d: %c\n", caractere, caractere);
```

No lugar do **%d** será impresso **97**

No lugar do **%c** será impresso o caractere correspondente ao número **97**

Saída

```
97: a
```

UTF - Unicode

- Um dos padrões mais utilizados da atualidade é o ***UTF-8*** (*8-bit Unicode Transformation Format*)

- ***UTF-8*** pode representar qualquer caractere universal padrão do Unicode, sendo também compatível com o ASCII

- Tabela Unicode:

https://www.utf8-chartable.de/unicode-utf8-table.pl

O que é uma string?

- ***string*** é um vetor de ***char***
- Declaração de uma ***string***:

charnome_string[tamanho]

- Exemplo: Declaração de uma string de 10 caracteres:

char minhaString[10]

O que é uma string?

- Uma particularidade da ***string*** é que seu último caractere é nulo (***NULL***)
- Último caractere: **'\0'** indica o final de uma string
- Pode ser chamado de terminador da string

string vs. char

- Na linguagem C deve-se ter muita atenção ao saber com qual tipo de dados está se trabalhando
- ***strings*** são representadas por aspas duplas **“ ”**
- ***char*** usam aspas simples **‘ ‘**

string vs. char

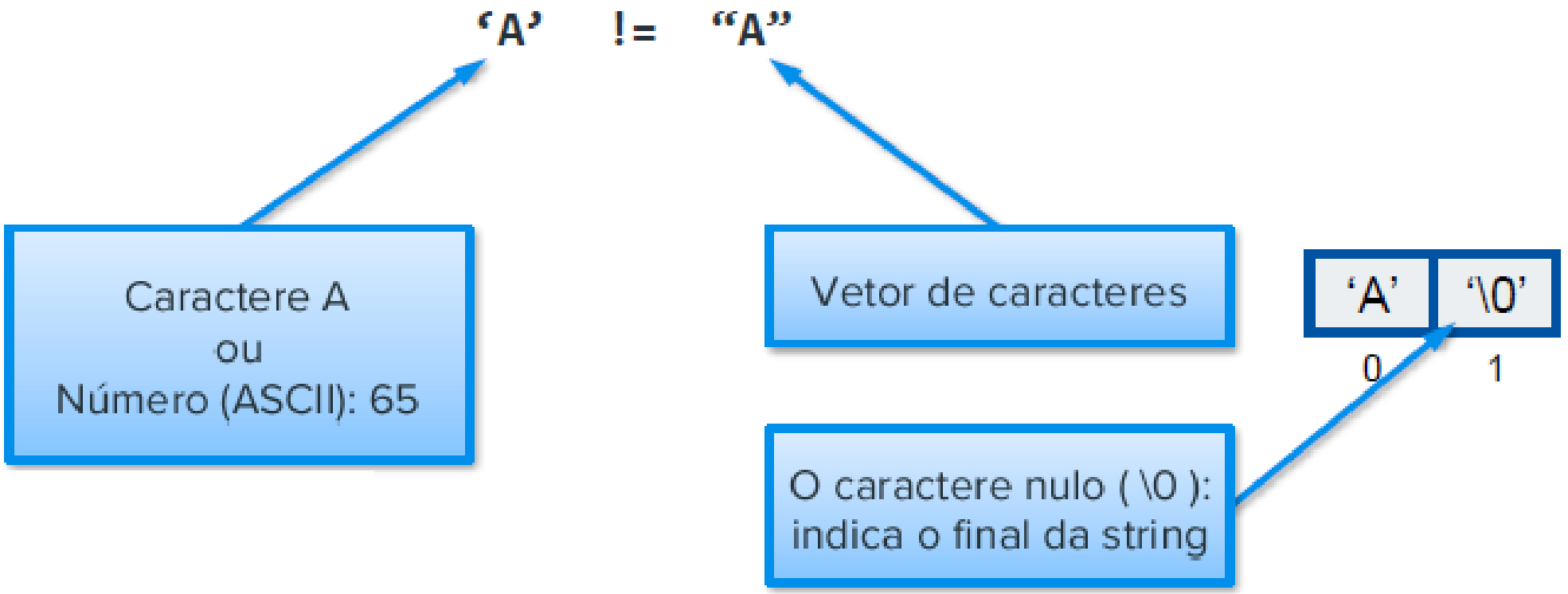
- Exemplos de ***char***:
  - ‘a’
  - ‘A’
  - ‘!’
  - ‘C’
  - ‘1’

- Exemplos de ***string***:
  - “Uma string”
  - “A”
  - “Linguagem C”
  - “Palavra”
  - “olá mundo!”

string vs. char

- Exemplos de ***char***:
  - ‘a’
  - ‘A’
  - ‘!’
  - ‘C’
  - ‘1’
- Exemplos de ***string***:
  - “Uma string”
  - “A”
  - “Linguagem C”
  - “Palavra”
  - “olá mundo!”

string vs. char

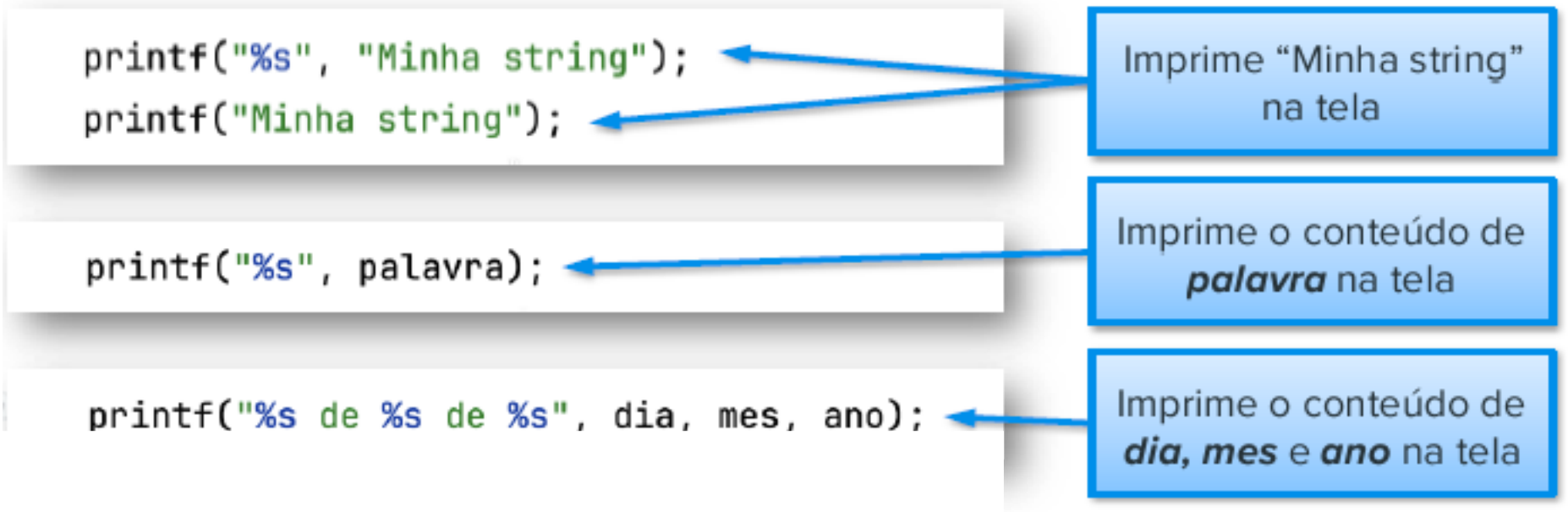


strings

- Impressão de ***strings***
  - Para imprimir uma string em um dispositivo de saída, usa-se a função printf():
  - Argumento de formatação da string: “%s”

strings

- Impressão de ***strings*** com a função ***printf()***



strings

- Impressão de ***strings*** com a função ***puts()***



strings

- Diferenças entre ***printf()*** e ***puts()***:
  - No final da string impressa, a função puts() adiciona automaticamente uma mudança de linha (\n)
  - Não é possível imprimir mais de uma string utilizando a função puts()

strings

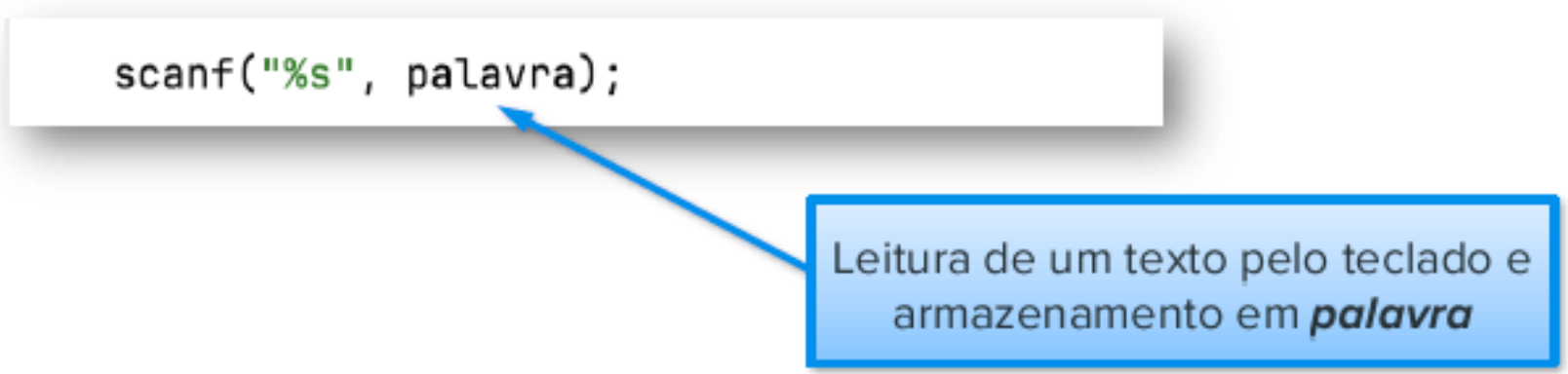
- Leitura de ***strings***

- A leitura de ***strings*** pode ser feita por meio dos seguintes comandos:

  - scanf()
  - gets()
  - fgets()

strings

- Leitura de ***strings***
- Para ler uma ***string*** em um dispositivo de entrada, usa-se a função ***scanf()***
- Argumento de formatação da string: ***"%s"***



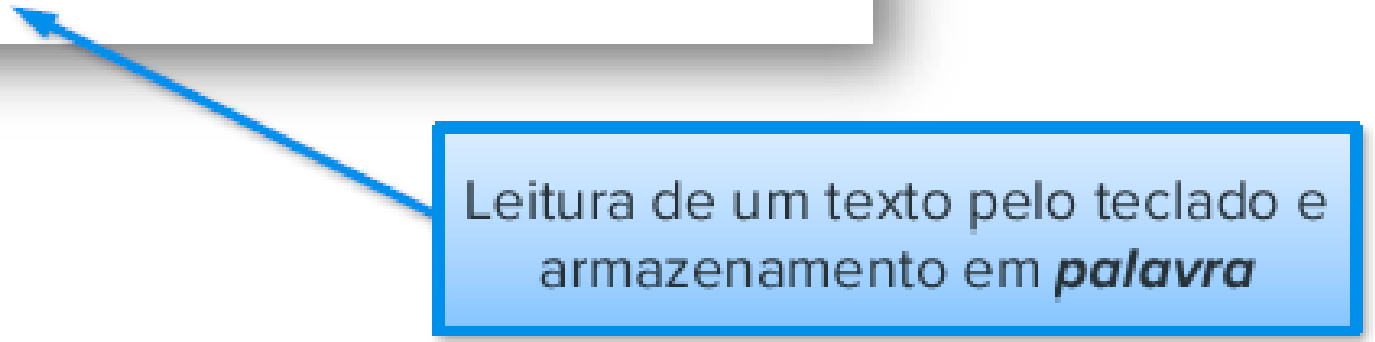
strings

- Leitura de ***strings***
- Problema do uso da função ***scanf()***:
  - A função scanf entende que a string acaba no primeiro espaço em branco ou quebra de linha
  - Desse modo, frases não serão lidas por esta função

strings

- Leitura de ***strings***
- Leitura de strings com ***gets()***:

```
gets(palavra);
```



strings

- Leitura de ***strings***
- Problema do uso da função ***gets()***:
  - Com o uso da função *gets*, se uma cadeia maior que o tamanho de criação da *string* for digitada, os caracteres excedentes serão transportados para a sequência de memória que não pertence à *string*, o que pode gerar comportamentos inesperados do programa

strings

- Leitura de ***strings***
- Leitura de strings com a função ***fgets()***

```
fgets(palavra, 10, stdin);
```

Leitura de um texto pelo teclado (***stdin***) com até 10 caracteres e armazenamento em ***palavra***

strings

- Leitura de ***strings***
- Problema do uso da função ***fgets()***:
  - Com o uso da função *fgets*, se uma cadeia maior que o tamanho de criação da *string* for digitada, apenas aquela quantidade de caracteres será transferida para a *string*, sendo os demais caracteres desprezados

strings

Ao final da frase pula para a primeira coluna da próxima linha.

```c
#include <stdio.h>
int main() {
    char frase[] = {'O', 'l', 'a', ' ', 'M', 'u', 'n', 'd', 'o', '!', '\n', '\0'};

    printf("%s", frase);
    return 0;
}
```

Saída (console):

```
Ola Mundo!

Process finished with exit code 0
```

strings

```c
#include <stdio.h>
int main() {
    char *frase = "Declaração como ponteiro para char.\n";

    printf("%s", frase);
    return 0;
}
```

Saída (console):

```
Declaração como ponteiro para char.

Process finished with exit code 0
```

strings

- Biblioteca ***string.h***

- É uma das bibliotecas padrão da linguagem C e tem como objetivo a manipulação de ***strings***

strings

- Biblioteca ***string.h***

- Principais funções:

  - **void *memchr(const void *str, int c, size_t n)**
  - Busca a primeira ocorrência do caractere **c** (unsigned char) nos primeiros **n** bytes da string apontada pelo argumento **str**

strings

- Biblioteca ***string.h***

- Principais funções:

  - **int memcmp(const void *str1, const void *str2, size_t n)**
  - Compara os primeiros **n** bytes de **str1** e **str2**

  - **void *memcpy(void *dest, const void *src, size_t n)**
  - Copia **n** caracteres de **src** para **dest**

strings

- Biblioteca ***string.h***

- Principais funções:

  - **void *memset(void*str, int c, size_t n)**
  - Copia o caractere **c** (unsignedchar) para os primeiros **n** caracteres da string apontada pelo argumento **str**

  - **char *strcat(char *dest, const char *src)**
  - Anexa a string apontada por **src** ao final da string apontada por **dest**

strings

- Biblioteca ***string.h***

- Principais funções:

  - **char *strncat(char *dest, const char *src, size_t n)**
  - Anexa a string apontada por **src** ao final da string apontada por **dest**, até um máximo de **n** caracteres

  - **char *strchr(const char *str, int c)**
  - Busca a primeira ocorrência do caractere **c** (unsignedchar) na string apontada pelo argumento **str**

strings

- Biblioteca ***string.h***

- Principais funções:

  - **int strcmp(const char *str1, const char *str2)**
  - Compara a string apontada por **str1** com a string apontada por **str2**

  - **int strncmp(const char *str1, const char *str2, size_t n)**
  - Compara no máximo os primeiros **n** bytes de **str1** e **str2**

strings

- Biblioteca ***string.h***

- Principais funções:

  - **char *strcpy(char *dest, const char *src)**
  - Copia a string apontada por **src** para **dest**

  - **char *strncpy(char *dest, const char *src, size_t n)**
  - Copia até **n** caracteres da string apontada por **src** para **dest**

strings

- Biblioteca ***string.h***

- Principais funções:

  - **size_t strlen(const char *str)**
  - Calcula o comprimento da string **str** até, mas não incluindo, o caractere nulo de terminação

  - **char *strtok(char *str, const char *delim)**
  - Divide a string "**str**" em uma série de tokens separados por "**delim**"

strings

- Bibliotecas e funções auxiliares

- ***ctype.h***

  - **int isalpha(int c)**
  - Verifica se o caractere passado é alfabético

  - **int isdigit(int c)**
  - Verifica se o caractere passado é um dígito decimal

strings

- Bibliotecas e funções auxiliares

- ***ctype.h***

  - **int islower(int c)**
  - Verifica se o caractere passado é uma letra minúscula

  - **int isupper(int c)**
  - Verifica se o caractere passado é uma letra maiúscula

strings

- Bibliotecas e funções auxiliares

- ***stdio.h***

  - **int sprintf(char *str, const char *format, ...)**
  - Envia a saída formatada para uma string

  - **int sscanf(const char *str, const char *format, ...)**
  - Lê a entrada formatada de uma string

strings

- E se for necessário fazer um vetor de ***strings***?
- É possível fazer uma matriz de ***char***?
- Como?

Matrix de char

- Exemplo

```c
#include <stdio.h>
#include <string.h>
int main() {
    int mes;
    char *meses[12][10];
    strcpy(meses[0], "Janeiro");
    strcpy(meses[1], "fevereiro");
    strcpy(meses[2], "março");
    strcpy(meses[3], "abril");
    strcpy(meses[4], "maio");
    strcpy(meses[5], "junho");
    strcpy(meses[6], "julho");
    strcpy(meses[7], "agosto");
    strcpy(meses[8], "setembro");
    strcpy(meses[9], "outubro");
    strcpy(meses[10], "novembro");
    strcpy(meses[11], "dezembro");

    for( mes = 0; mes < 12; mes++) {
        printf("Mês %d = %s\n", mes+1, meses[mes]);
    }
    return 0;
}
```

Saída (console):

```
Mês 1 = Janeiro
Mês 2 = fevereiro
Mês 3 = março
Mês 4 = abril
Mês 5 = maio
Mês 6 = junho
Mês 7 = julho
Mês 8 = agosto
Mês 9 = setembro
Mês 10 = outubro
Mês 11 = novembro
Mês 12 = dezembro

Process finished with exit code 0
```

Matrix de char

- Exemplo com ponteiros

```c
#include <stdio.h>
#include <string.h>
int main() {
    int mes;
    char *meses[12];
    meses[0] = "Janeiro";
    meses[1] = "fevereiro";
    meses[2] = "março";
    meses[3] = "abril";
    meses[4] = "maio";
    meses[5] = "junho";
    meses[6] = "julho";
    meses[7] = "agosto";
    meses[8] = "setembro";
    meses[9] = "outubro";
    meses[10] = "novembro";
    meses[11] = "dezembro";

    for( mes = 0; mes < 12; mes++) {
        printf("Mês %d = %s\n", mes+1, meses[mes]);
    }
    return 0;
}
```

Saída (console):

```
Mês 1 = Janeiro
Mês 2 = fevereiro
Mês 3 = março
Mês 4 = abril
Mês 5 = maio
Mês 6 = junho
Mês 7 = julho
Mês 8 = agosto
Mês 9 = setembro
Mês 10 = outubro
Mês 11 = novembro
Mês 12 = dezembro

Process finished with exit code 0
```

Exercícios

1. Escreva um programa que recebe quatro strings(sem utilizar o scanf()) que representam números inteiros, converte as string sem inteiros, soma os valores e imprime o total dos quatro valores.

2. Escreva um programa que recebe quatro strings(sem utilizar o scanf()) que representam números reais (float), converte as string sem reais (float), soma os valores e imprime o total dos quatro valores.

3. Escreva um programa que recebe uma linha de texto no vetor de char texto[100]. Produza a linha em letras maiúsculas e em letras minúsculas.

Obrigada pela sua participação, nos vemos na próxima aula!

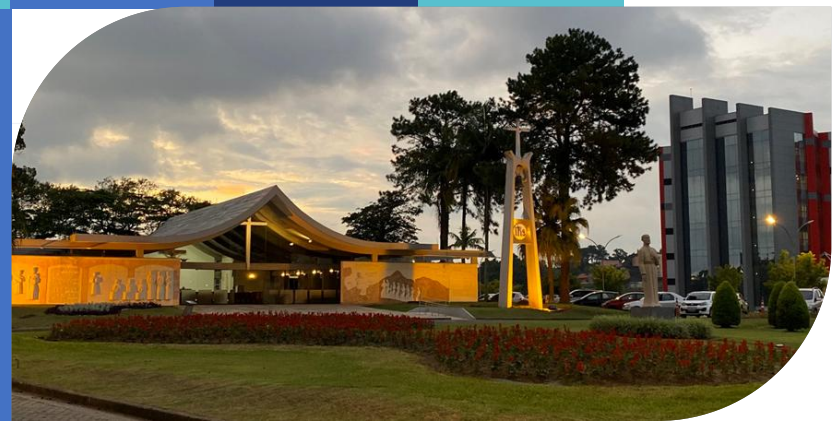
:)

JESUÍTAS BRASIL









FEI edu.br

JESUÍTAS BRASIL